fetch_data

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN

Giảng viên hướng dẫn: Kim Ngọc Bách

Học phần : Lập trình Python

Nhóm : 06

Thành viên nhóm : Triệu Việt Anh – B23DCCN054

Hoàng Mạnh Hiệp – B23DCCN292

Lương Phương Quang – B21DCCN627

Hà Nội – 2025

Phần I: Thu tập dữ liệu cầu thủ

Câu I.1: Thu thập dữ liệu thống kê của tất cả các cầu thủ có số phút thi đấu nhiều hơn 90 phút tại giải bóng đá ngoại hạng anh

1. **Mục đích:** Chương trình cào và hợp nhất dữ liệu thống kê đa của cầu thủ Premier League (2024-2025) từ FBref. Dữ liệu sau đó được lọc (chỉ giữ cầu thủ > 90 phút) và lưu vào SQLite/CSV.
2. **Khai báo thư viện**

   Chương trình bắt đầu bằng việc khai báo các thư viện được phân loại thành 4 nhóm chức năng chính:

   - **Hệ thống và tiện ích:**
     - time, random (dùng cho hàm human_pause để chống bot)
     - re (Regular Expressions - dùng để làm sạch dữ liệu text)
   - **Lưu trữ dữ liệu:** sqlite3 (để ghi dữ liệu vào CSDL SQLite theo yêu cầu).
   - **Xử lý dữ liệu:** pandas as pd (dùng để cấu trúc, hợp nhất, và lọc dữ liệu; đây là thư viện xử lý chính).
   - **Web Scraping:**
     - **BeautifulSoup, Comment (từ bs4):** Dùng để phân tích (parse) mã HTML. Việc import Comment là **cực kỳ quan trọng** và là chìa khóa của bài này, vì fbref.com ẩn bảng trong comment.
     - **Selenium:** Toàn bộ thư viện này (từ webdriver đến WebDriverWait) được dùng để tự động hóa trình duyệt (Chrome). Điều này là bắt buộc vì fbref.com cần Selenium để "lấy" được mã HTML chứa các comment đó.
3. **Cấu hình toàn cục**

   Đây là các biến cấu hình, giúp code dễ đọc và dễ bảo trì:

   - **DB_FILE, OUT_TABLE:** Định nghĩa tên file CSDL (football.sqlite) và tên bảng (pl_stats_clean) sẽ được tạo.
   - **URL, SS:** Định nghĩa URL gốc của giải đấu và chuỗi mùa giải (2024-2025).
   - **STAT_PAGES: Đây là một cấu trúc dữ liệu cốt lõi.** Nó là một dictionary (từ điển) ánh xạ 6 loại thống kê mà bạn muốn cào. Mỗi mục chứa:
     - Tên hậu tố URL (ví dụ: passing).
     - ID của bảng HTML (ví dụ: stats_passing).
     - ID của div chứa bảng đó (ví dụ: all_stats_passing). Cấu trúc này giúp hàm main có thể lặp (loop) qua 6 trang một cách gọn gàng.
   - **MIN_MINUTES_THRESHOLD:** Ngưỡng 90 phút theo yêu cầu đề bài.
   - **WAIT_SEC:** Thời gian chờ cơ bản, dùng cho human_pause.
4. **Các hàm khởi tạo và hỗ trợ**

Đây là các hàm tiện ích nhỏ, phục vụ cho các tác vụ cụ thể:

- **build_driver():** Khởi tạo Selenium Chrome driver.
  - **Chi tiết quan trọng:** opts.add_argument("--disable-blink-features=AutomationControlled"). Đây là một kỹ thuật "tàng hình" (stealth) quan trọng, giúp che giấu việc trình duyệt đang bị điều khiển bởi Selenium, nhằm qua mặt các cơ chế phát hiện bot cơ bản của fbref.com.
- **human_pause():** Hàm này thông minh hơn time.sleep(1.5) cố định. Bằng cách dùng random.uniform(a, b), nó tạo ra thời gian chờ ngẫu nhiên, mô phỏng hành vi của người dùng thật và **giúp chống rate-limit (chặn IP)**.
- **db_file_short():** Hàm tiện ích để in tên file cho đẹp.
- **clean_nation_value():** Dùng Regex (re.search) để rút trích mã quốc gia (ví dụ: "en ENG" -> "ENG"). Đây là bước làm sạch dữ liệu (data cleaning) cần thiết.

5. **Bóc tách dữ liệu**

Đây là hai hàm quan trọng nhất giải quyết vấn đề của fbref.com:

- **Hàm extract_table_html_from_container():**
  - **Vấn đề:** fbref.com không hiển thị bảng (<table>) trực tiếp trong HTML. Họ giấu nó bên trong một thẻ chú thích (``) để chống cào dữ liệu.
  - **Giải pháp:**
    - Hàm này không tìm <table> ngay. Nó tìm div chứa bảng (container_id).
    - Nó lấy innerHTML của div đó.
    - Nó dùng BeautifulSoup duyệt qua tất cả các phần tử con, và **chỉ tìm các phần tử là Comment** (isinstance(t, Comment)).
    - Nếu một comment có chứa ID của bảng (expected_table_id in c), nó sẽ bóc tách nội dung của comment đó, và *lúc này* mới tìm <table> bên trong.
  - **Kết quả:** Trả về mã HTML của bảng đã bị ẩn.
- **Hàm parse_table_html():**
  - Hàm này nhận mã HTML của bảng (từ hàm trên) và chuyển nó thành dữ liệu có cấu trúc (danh sách các dictionary).
  - Nó đọc thead (đầu bảng) để lấy tiêu đề cột (ưu tiên data-stat vì nó ổn định).
  - Nó lặp qua từng hàng tr trong tbody (thân bảng).
  - **Logic tránh trùng lặp:**
    - Nếu là trang standard, nó lấy tất cả thông tin (tên, CLB, quốc tịch, vị trí, v.v.).
    - Nếu là các trang khác (shooting, passing...), nó dùng REDUNDANT_COLUMNS_NON_STANDARD để **bỏ qua** các cột thông tin chung đã lấy ở trang standard (như 'age', 'position'), và chỉ giữ lại các chỉ số thống kê của riêng trang

đó. Điều này rất quan trọng để chuẩn bị cho việc hợp nhất (merge) dữ liệu sau này.

6. **Xử lý dữ liệu**
    - Đây là các hàm dùng pandas để xử lý dữ liệu sau khi cào về:
    - **Vấn đề:** Cần lọc cầu thủ có tổng phút > 90, nhưng cột "phút" có thể có tên khác nhau và đang ở dạng text (ví dụ: "1,200")
    - **Giải pháp:**
        - Hàm này **tự động tìm cột phút** (minute_col) bằng cách tìm tên cột chứa "minute" nhưng không chứa "90" (vì "90s" là chỉ số *mỗi 90 phút*, không phải tổng phút).
        - Nó dùng pd.to_numeric(..., errors='coerce') kết hợp với .str.replace(",", "") để chuyển cột phút từ text (văn bản) sang dạng số (numeric) một cách an toàn.
        - Cuối cùng, nó trả về một DataFrame mới đã được lọc (df[df[minute_col] > threshold]).
7. **Lưu trữ dữ liệu**

    - **Hàm write_sqlite():**
        - Hàm này nhận danh sách **records** (kết quả cuối cùng) và ghi vào SQLite.
        - **Tính năng linh hoạt:** Nó tự động xây dựng câu lệnh **CREATE TABLE** bằng cách duyệt qua tất cả các cột có trong **records**. Điều này có nghĩa là nếu bạn thêm/bớt chỉ số, CSDL sẽ tự động điều chỉnh theo.
        - Nó dọn dẹp và sắp xếp lại các cột (đẩy 'id', 'player', 'club' lên đầu).

    - Nó dùng **cur.executemany()** với các tham số (?). Đây là cách ghi dữ liệu hàng loạt (batch insert) **nhanh và an toàn** (chống SQL injection) vào CSDL.
8. **Thực thi (Hàm main)**
    - **Khởi tạo:** gọi **build_driver()**. Tạo một **list_of_dataframes** rỗng (để chứa 5 bảng phụ) và **standard_df = None** (để chứa bảng chính).
    - **Vòng lặp Scraping (trên STAT_PAGES):**
        - Lặp qua 6 trang (standard, shooting, ...).
        - Với mỗi trang: driver.get(url), chờ (WebDriverWait), rồi human_pause().
        - Gọi **extract_table_html_from_container()** -> **parse_table_html()** để lấy rows.
        - Chuyển rows thành page_df (Pandas DataFrame).
    - **Chiến lược hợp nhất:** Đây là phần xử lý dữ liệu thông minh.
        - Nếu là **standard:** Bảng này được coi là "bảng gốc" (base table). Nó đổi tên các cột thống kê (ví dụ: Gls -> standard__Gls) và lưu vào **standard_df**.
        - Nếu là **các trang khác:** Bảng này là "bảng phụ". Nó cũng đổi tên các cột (ví dụ: Sh -> shooting__Sh) và thêm **page_df** vào **list_of_dataframes**.

- **Đóng trình duyệt:** driver.quit() (sau khi try...finally đảm bảo dù lỗi cũng đóng).
- **Hợp nhất dữ liệu:**
  - Nó lấy **standard_df** làm bảng chính
  - Nó lặp qua **list_of_dataframes** (5 bảng phụ) và gộp (merge) từng bảng một vào bảng chính
  - **Chi tiết quan trọng:** pd.merge(..., on=["player", "club"], how="outer").
    - **on = ["player", "club"]:** Khóa hợp nhất.
    - **how="outer" (Outer Join):** Đảm bảo rằng mọi cầu thủ (kể cả thủ môn không có trong bảng 'shooting') đều được giữ lại. Các chỉ số không tồn tại sẽ tự động được điền là NaN (sau này được chuyển thành "N/a").
- **Hoàn thiện và lưu trữ:**
  - Gọi **filter_dataframe_by_minutes()** để lọc.
  - **master_df.fillna("N/a"):** Làm sạch các ô trống (kết quả của outer merge).
  - **write_sqlite():** Lưu vào CSDL.
  - **df_final.to_csv():** Lưu thêm ra file CSV.

Câu I.2: Thu thập dữ giá chuyển nhượng cầu thủ

1. **Mục đích:** Chương trình cào dữ liệu giá chuyển nhượng của cầu thủ Premier League (2024-2025) từ footballtransfers.com. Dữ liệu này được cào dựa trên danh sách cầu thủ đã lọc và lưu vào một bảng riêng trong SQLite/CSV.
2. **Khai báo thư viện**

   Chương trình bắt đầu bằng việc khai báo các thư viện được phân loại thành 4 nhóm chức năng chính:

   - **Hệ thống tiện tích:** sys (dùng để cấu hình stdout xử lý tiếng Việt), time (dùng để sleep, chống rate-limit), re (Regular Expressions - dùng để tạo "slug").
   - **Xử lý văn bản:** unidecode (một thư viện bên ngoài, dùng để chuyển đổi văn bản có dấu, ví dụ: "Ødegaard" -> "Odegaard", một bước bắt buộc để tạo URL)
   - **Lưu trữ và xử lý dữ liệu:** sqlite3 (để đọc CSDL từ Phần I.1) và pandas as pd (dùng để ghi dữ liệu ra CSDL và CSV một cách hiệu quả).
   - **Web Scraping;**

     - **requests:** Thư viện chính để gửi yêu cầu HTTP (tải trang web). Nó nhanh và nhẹ hơn Selenium, phù hợp với trang web tĩnh này.
     - **BeautifulSoup (từ bs4):** Dùng để phân tích (parse) mã HTML trả về từ requests

3. **Cấu hình toàn cục:**

Các biến này định nghĩa các tham số cố định, giúp dễ dàng thay đổi khi cần:

- **DB_FILE:** Chỉ định file SQLite (phải giống với Phần I.1 để đọc dữ liệu).
- **STATS_TABLE:** Tên bảng (**pl_stats_clean**) mà Phần I.1 đã tạo. Script này sẽ đọc từ bảng này.
- **TARGET_TABLE:** Tên bảng (**player_transfers**) mà script này sẽ **ghi** kết quả vào.
- **CSV_OUTPUT_FILE:** Tên file CSV đầu ra.
- **BASE_URL:** URL gốc của trang footballtransfers.com.
- **HEADERS:** User-Agent: Khai báo HEADERS với một User-Agent giả mạo (Chrome 120) là điều **bắt buộc**. Nếu không có nó, nhiều trang web sẽ nhận ra đây là một script Python và chặn yêu cầu (trả về lỗi 403 Forbidden).

4. Cấu trúc lớp

Toàn bộ logic của script được đóng gói trong một class (lớp) tên là **TransferScraper.**

- **__init__() (Hàm khởi tạo)**
    - Đây là hàm đầu tiên khi chạy đối tượng **TransferScraper** được tạo.
    - Nó lưu trữ các biến cấu hình (như tên file, tên bảng).
    - Nó tạo ra **self.sesion = requests.Session().**
        - Việc dùng Session thay vì **requests.get()** đơn lẻ có lợi ích lớn: nó giúp **duy trì cookies** và các HEADERS (được cập nhật ngay sau đó) qua tất cả các yêu cầu. Điều này khiến máy chủ tin rằng tất cả các yêu cầu đều đến từ cùng một người dùng, giúp **chống rate-limit** hiệu quả hơn.
    - Nó cũng tạo một danh sách rỗng **self.results** để lưu kết quả cào được.
- **_init_db() (Hàm khởi tạo CSDL)**
    - **Mục đích:** Dọn dẹp bảng cũ trước khi chạy.
    - Nó kết nối đến CSDL và thực thi DROP TABLE IF EXISTS... để xóa bảng player_transfers cũ (nếu có).
    - Sau đó, nó tạo lại bảng **player_transfers** với cấu trúc 4 cột.
    - Dấu gạch dưới _ ở đầu tên hàm **(_init_db)** là một quy ước trong Python, ngụ ý rằng đây là một hàm "nội bộ", chỉ nên được dùng bên trong class.
- **_get_players() (Hàm lấy danh sách cầu thủ)**
    - Mục đích: Đọc danh sách cầu thủ cần tìm giá từ kết quả của Phần I.1.
    - Nó kết nối đến **DB_FILE** và thực thi câu lệnh **SQL SELECT DISTINCT player FROM [pl_stats_clean]....**
    - Nó lưu danh sách tên cầu thủ vào **self.players_to_scrape**
- **_make_slug() (Hàm tạo "slug" URL)**

- **Vấn đề:** Tên cầu thủ như "Martin Ødegaard" cần được chuyển thành "martin-odegaard" để đặt vào URL.
- **Giải pháp**:

    - **unidecode(name.lower()):** Chuyển tên về chữ thường và **loại bỏ tất cả dấu** (ví dụ: "Ø" -> "o", "é" -> "e").
    - **re.sub(r"[^a-z0-9\s-]", "", name):** Dùng Regex **loại bỏ mọi ký tự không phải** là chữ cái (a-z), số (0-9), khoảng trắng, hoặc dấu gạch ngang.
    - **re.sub(r"\s+", "-", name.strip()):** Thay thế một hoặc nhiều khoảng trắng bằng một dấu gạch ngang duy nhất.

- **Kết quả:** Trả về một "slug" sạch, sẵn sàng cho URL.

- **_fetch_player_value() (Hàm cào dữ liệu chính)**

    - **Vấn đề:** Đôi khi có nhiều cầu thủ trùng tên (ví dụ: 2 cầu thủ "John Smith"). Trang web có thể xử lý bằng cách thêm -1, -2 vào cuối URL.
    - **Giải pháp:**

        - Tạo "slug" (ví dụ: john-smith)
        - Tạo một danh sách 3 URL tiềm năng: .../john-smith, .../john-smith-1, và .../john-smith-2
        - Lặp qua 3 URL này:

            - Dùng **self.session.get(url)** (đã có User-Agent từ __init__) để tải trang
            - **time.sleep(0.3):** Tạm dừng ngắn để **chống rate-limit**.
            - Nếu **res.status_code != 200** (ví dụ: lỗi 404 Not Found), nó sẽ continue (bỏ qua và thử URL tiếp theo).
            - Dùng **BeautifulSoup** để phân tích res.text
            - **Bóc tách dữ liệu:** Dùng **soup.select_one(".player-value .player-tag")**. Đây là một **CSS Selector** tìm thẻ có class player-tag nằm bên trong thẻ có class player-value.
            - Nếu tìm thấy cả giá trị và CLB, nó trả về tuple dữ liệu và **thoát khỏi vòng lặp**.

    - Nếu thử cả 3 URL mà vẫn không tìm thấy, hàm sẽ trả về (player, "N/a", "N/a", "N/a")

- **_write_results() (Hàm ghi kết quả)**

    - **Mục đích:** Lưu danh sách self.results (đã thu thập) vào CSDL và CSV.
    - **Giải pháp (Sử dụng Pandas):** Đây là cách làm rất hiệu quả.

        - Chuyển đổi danh sách **self.results** thành một **pandas.DataFrame.**
        - **Gọi df.to_sql(self.target_table, ...):** Pandas tự động kết nối và ghi toàn bộ DataFrame vào bảng SQLite

- **Gọi df.to_csv(self.csv_output, ...):** Pandas ghi DataFrame ra file CSV. encoding="utf-8-sig" đảm bảo file CSV mở đúng tiếng Việt trên Excel.

5. **Thực thi**
   - **run()**
     - **_init_db():** Dọn dẹp CSDL.
     - **_get_players():** Lấy danh sách cầu thủ
     - Lặp qua **self.players_to_scrape**, gọi **_fetch_player_value()** cho từng cầu thủ và append kết quả vào **self.results**
     - **_write_results():** Lưu kết quả cuối cùng
   - **if __name__ == "__main__":**
     - Đây là **điểm bắt đầu** thực thi của script
     - Nó tạo ra một đối tượng từ lớp TransferScraper: **scraper = TransferScraper(...)**
     - Cuối cùng, nó gọi **scraper.run()** để bắt đầu toàn bộ quy trình

❖ **Kết quả:** Đưa ra file sqlite chữa hai bảng pl_stat_clean và pl_transfers

Phần II: Tra cứu cầu thủ

Câu II.1: Xây dựng API tra cứu dữ liệu cầu thủ bằng Flask

1. **Mục đích:** Đề tài sử dụng Python Flask kết hợp với SQLite để xây dựng RESTful API phục vụ tra cứu dữ liệu bóng đá được lưu trữ trong bảng.
2. **Khai báo thư viện**
   Code bắt đầu bằng việc nhập các thư viện được chia làm 3 nhóm chính:
   - **Web Framework (Cốt lõi):**
     - **flask:** Thư viện micro-framework chính để xây dựng máy chủ web.
     - **Flask:** Lớp (class) chính để tạo ứng dụng.
     - **request:** Đối tượng toàn cục, dùng để truy cập dữ liệu gửi đến (ví dụ: request.args.get("name")).
     - **jsonify:** Hàm để chuyển đổi dict (từ điển) Python thành phản hồi JSON chuẩn cho API.
     - **render_template_string:** Dùng để render (biên dịch) một chuỗi HTML (thay vì một file .html).
   - **API Framework (Tiện ích):**
     - flask_restful (Api, Resource): Một phần mở rộng (extension) của Flask, giúp tổ chức code API theo các "Resource" (Tài nguyên) một cách sạch sẽ và chuyên nghiệp.
   - **Xử lý Dữ liệu:**

    - **sqlite3:** Dùng để kết nối với CSDL SQLite
    - **pandas as pd:** Được dùng như một công cụ truy vấn (query) và định dạng (format) dữ liệu rất mạnh mẽ.
3. **Cấu hình và khởi tạo**
    - **app = Flask(__name__):** Khởi tạo ứng dụng web Flask
    - **api = Api(app):** Khởi tạo phần mở rộng flask_restful và "gắn" nó vào ứng dụng app
    - **DB_PATH, TABLE_NAME:** Định nghĩa các hằng số cho CSDL (ở đây là football.sqlite)
4. **Hàm hỗ trợ**
    - **Hàm fetch_data(query, params=()):**
        - Đây là một hàm hỗ trợ (helper) cực kỳ quan trọng và được thiết kế tốt.
        - **Mục đích:** Tách biệt logic truy vấn CSDL ra khỏi logic xử lý API.
        - **Hành động:**
            - Kết nối đến DB_PATH.
            - **Sử dụng pd.read_sql_query():** Hàm thực thi câu SQL và tải thẳng kết quả vào một DataFrame của Pandas.
            - **Bảo mật (Rất quan trọng):** Hàm này sử dụng params=params. Khi truyền tham số (ví dụ: ("%Son%",)) vào read_sql_query, Pandas sẽ tự động "làm sạch" (sanitize) dữ liệu đầu vào. Điều này ngăn chặn hoàn toàn lỗi bảo mật SQL Injection.
            - **Xử lý lỗi:** Bọc toàn bộ logic trong try...except. Nếu có lỗi (ví dụ: gõ sai tên bảng), nó sẽ trả về (None, str(e)) (Không có data, và thông báo lỗi).
    - **Kết quả:** Trả về (DataFrame, Lỗi)
5. **Logic API**
    - **Lớp SearchPlayer(Resource):**
        - **Mục đích:** Tạo ra endpoint /player để tìm kiếm cầu thủ.
        - Kế thừa từ Resource của flask_restful
        - **Phương thức get(self):** Phương thức này sẽ tự động được gọi khi có một yêu cầu GET đến endpoint này
        - **Logic**
            - **request.args.get("name"):** Lấy tham số name từ URL (ví dụ: /player?name=Son).
            - **Kiểm tra đầu vào (Validation):** if not player_name... Nếu người dùng không cung cấp name, trả về lỗi 400 (Bad Request).
            - **Truy vấn:** Gọi hàm **fetch_data** với câu lệnh LIKE ? và tham số f"%{player_name}%" (dấu % nghĩa là "tìm kiếm chứa" tên đó).
            - **Xử lý kết quả:**
                - **if df is None:** Lỗi CSDL -> Trả về lỗi 500 (Internal Server Error)
                - **if df.empty:** Không tìm thấy -> Trả về lỗi 404 (Not Found)
                - **Thành công:** Dùng df.to_dict(orient="records") để chuyển DataFrame (bảng) thành một danh sách JSON (list of objects) và trả về với mã 200 (OK)

- **Lớp SearchClub(Resource)**
    - Logic y hệt như PlayerLookup, nhưng dành cho club và truy vấn vào cột club.

6. **Đăng kí API**
    - **api.add_resource**(PlayerLookup, "/player")
    - **api.add_resource**(ClubLookup, "/club")
    - **Mục đích:**
        - Bất kỳ yêu cầu nào đến URL /player sẽ do lớp PlayerLookup xử lý.
        - Bất kỳ yêu cầu nào đến URL /club sẽ do lớp ClubLookup xử lý
7. **Giao diện người dùng UI**
    - **Hàm ui_html():**
        - **Mục đích:** Cung cấp một giao diện web đơn giản để người dùng (không biết kỹ thuật) có thể sử dụng API.
        - **Kỹ thuật:**
            - **@app.route("/ui"):** Sử dụng một route Flask *thông thường* (không phải flask_restful) để tạo trang /ui
            - **render_template_string(html):** Thay vì tạo file index.html riêng biệt, toàn bộ HTML, CSS, và JavaScript được nhúng (embedded) vào một chuỗi Python và render trực tiếp. Điều này giúp toàn bộ ứng dụng nằm gọn trong 1 file duy nhất.
        - **Phân tích Giao diện (HTML/CSS/JS):**
            - **HTML:** Cấu trúc rất cơ bản, có 2 ô input (cho player, club) và 2 nút button
            - **CSS (Bootstrap):** Tải thư viện Bootstrap 5 từ CDN để giao diện trông đẹp và chuyên nghiệp ngay lập
            - **CSS (Custom):** Thêm một số CSS tùy chỉnh (<style>...) để tạo giao diện "Dark Mode" (nền tối, chữ trắng) rất đẹp mắt
            - **JavaScript**:
                - **searchPlayer() và searchClub():** Hai hàm này được gọi khi nhấn nút
                - **fetch(...):** Sử dụng API fetch (của trình duyệt) để gọi đến 2 endpoint (/player và /club) đã tạo ở trên.
                - **showResult(data):** Hàm này nhận dữ liệu JSON trả về từ API và "vẽ" (render) kết quả ra màn hình bằng cách tự tạo mã HTML cho bảng (<table>). Nó cũng xử lý cả trường hợp lỗi (hiển thị alert-danger).
8. **Thực thi**
    - **Mục đích:** Đây là điểm khởi chạy server
    - **if __name__ == "__main__":** Đảm bảo rằng app.run() chỉ được gọi khi chạy file này trực tiếp (ví dụ: python app.py).
    - **print(...):** In ra các đường link hữu ích cho người chạy server
    - **app.run(debug=True):** Khởi động máy chủ web.
        - debug=True:

- Hiển thị lỗi chi tiết trên trình duyệt.
- Tự động khởi động lại (reload) server mỗi khi lưu file code.

Câu II.2:

1. **Mục đích:** Chương trình sử dụng thư viện requests để gửi HTTP request đến API Flask và Pandas để lưu kết quả ra file CSV.
2. **Khai báo thư viện**
   Code bắt đầu bằng việc nhập các thư viện được chia làm 5 nhóm:
   - **Tạo Giao diện Dòng lệnh (Cốt lõi):**
     - **argparse:** Nó tự động xử lý việc "phân tích" (parse) các tham số mà người dùng gõ vào (ví dụ: --name "Son").
   - **Giao tiếp Mạng:**
     - **requests:** Dùng để gửi yêu cầu HTTP (gọi API) đến server Flask.
   - **Xử lý Dựa liệu:**
     - **json:** Dùng để in (dump) kết quả JSON ra màn hình cho đẹp (indent=4)
     - **pandas as pd:** Dùng để chuyển đổi dữ liệu JSON (list of dicts) nhận về thành DataFrame và lưu ra file CSV.
   - **Thông tin Hệ thống:**
     - **platform, os:** Tự động mở file CSV bằng Excel/phần mềm mặc định chỉ khi người dùng đang chạy script trên Windows (platform.system() == "Windows" và os.startfile).
3. **Cấu hình toàn cục**
   - **BASE_URL = "http://localhost:5000":**
     - Định nghĩa địa chỉ của server Flask.
     - Biến này phải khớp với địa chỉ mà server Flask đang chạy.
4. **Xử lý đầu vào**
   - **Hàm parse_arguments():**
     - **Mục đích:** Sử dụng argparse để định nghĩa và lấy các tham số từ dòng lệnh
     - **parser.add_argument("--name", ...):** Thêm một tùy chọn (argument) là –name
     - **parser.add_argument("--club", ...):** Thêm một tùy chọn là –club
     - **Xử lý lỗi (Validation):** if not args.name and not args.club:. Đây là một bước kiểm tra đầu vào rất tốt. Nếu người dùng chạy script mà không cung cấp --name hoặc --club, chương trình sẽ in thông báo hướng dẫn và exit() (thoát) ngay lập tức.
     - **Kết quả:** Trả về một đối tượng args, nơi bạn có thể truy cập các giá trị (ví dụ: args.name).
5. **Xử lý Logic**
   - **Mục đích:** Một hàm "chuẩn bị" rất gọn gàng. Dựa trên args nhận được, nó quyết định xem sẽ gọi API nào và tên file output là gì.
   - **Logic:**

- Nếu args.name tồn tại (người dùng gõ --name ...), nó trả về 3 giá trị: endpoint (/player), tham số ({"name": ...}), và tên file (ví dụ: "Son Heung-min" -> Son_Heung_min.csv).
- Tương tự cho args.club

- Hàm send_request()
  - **Mục đích:** Đóng gói toàn bộ logic gọi API vào một hàm duy nhất.
  - Nó xây dựng URL đầy đủ (ví dụ: http://localhost:5000/player).
  - **response = requests.get(url, params=params):** Gửi request. requests sẽ tự động chuyển params (ví dụ: {"name": "Son"}) thành URL query (ví dụ: .../player?name=Son)
  - **Xử lý lỗi:** response.raise_for_status(). Dòng này sẽ tự động "văng" lỗi (raise exception) nếu API trả về các mã lỗi như 404 (Not Found), 500 (Server Error).
  - **except requests.exceptions.RequestException as e:** Bắt *mọi* lỗi liên quan đến requests (bao gồm lỗi 404, 500 từ dòng trên, hoặc lỗi mạng, DNS...) và in ra thông báo thân thiện rồi exit().
- **Hàm save_csv()**
  - **Mục đích:** Lưu kết quả vào file CSV và tự động mở file đó.
  - **df = pd.DataFrame(data):** Chuyển list (JSON) thành DataFrame.
  - df.to_csv(..., encoding="utf-8-sig"): utf-8- đảm bảo file CSV mở trên Excel (Windows) không bị lỗi font tiếng Việt hoặc các ký tự đặc biệt (như Ø).
  - **Xử lý lỗi (Cực kỳ hay): except PermissionError:** Đây là một lỗi rất phổ biến: người dùng chạy script, file CSV được mở trong Excel, họ quên đóng Excel và chạy lại script. Script sẽ bị lỗi vì không thể ghi đè file đang mở. Bằng cách except đúng lỗi PermissionError.
  - **Tính năng Thêm (UX):** Đoạn code kiểm tra platform.system() == "Windows" và dùng os.startfile(filename) giúp xem kết quả ngay lập tức.

6. **Thực Thi**
   - **args = parse_arguments():** Lấy lệnh từ người dùng
   - **endpoint, params, filename = build_request(args):** Quyết định phải làm gì
   - **data = send_request(endpoint, params):** Thực thi (gọi API)
   - **print(json.dumps(data, ...)):** In kết quả JSON thô ra terminal cho người dùng xem ngay. ensure_ascii=False giúp hiển thị tiếng Việt và tên riêng (Ødegaard) đúng ngay trên terminal
   - **Kiểm tra kết quả**:
     - **if isinstance(data, list) and len(data) > 0:** Kiểm tra xem data trả về có phải là một *danh sách* (list) và có dữ liệu không.
     - Nếu đúng (tức là API trả về danh sách cầu thủ/CLB), nó gọi save_csv().
     - Nếu sai (tức là API trả về lỗi như {"error": ...} hoặc {"message": "..."}), nó sẽ báo "Không có dữ liệu để lưu".

- **Điểm bắt đầu (if __name__ == "__main__":)**
    - Gọi main() để bắt đầu toàn bộ quy trình.

❖ **Kết quả:** Cung cấp cho người dùng tra cứu thông tin trên web, terminal và tự động lưu kết quả ra file CSV.

Phần III: Phân tích dữ liệu cầu thủ

I. Giới thiệu

Báo cáo này trình bày một quy trình phân tích dữ liệu toàn diện được thực hiện trên bộ dữ liệu thống kê cầu thủ (player_stats.csv) và đội bóng (squad_stats.csv). Mục tiêu là trích xuất các thông tin chi tiết từ cấp độ đội bóng đến việc định giá và phân loại vai trò của từng cầu thủ.

Các kỹ thuật chính được sử dụng bao gồm:

1. **Thống kê mô tả** (Sử dụng Pandas) để tổng hợp hiệu suất đội.
2. **Mô hình hóa tương đối** (Định giá Z-Score) để đánh giá giá trị cầu thủ.
3. **Học không giám sát (Clustering)** (Thuật toán K-Means) để phân nhóm cầu thủ.
4. **Giảm chiều dữ liệu (Dimensionality Reduction)** (Thuật toán PCA) để trực quan hóa các nhóm.

II. Phân tích Thống kê & Hiệu suất Đội (Mục III.1)

1. Trung bình, Trung vị và Độ lệch chuẩn theo Đội

- Lý giải cách làm:
  Mục tiêu là tính toán các chỉ số thống kê trung tâm (mean - trung bình), phân vị (median - trung vị) và độ phân tán (std - độ lệch chuẩn) cho mọi chỉ số của cầu thủ, nhưng được nhóm theo từng đội ('Squad'). Điều này cho phép chúng ta hiểu được đặc điểm trung bình và mức độ đồng đều của các cầu thủ trong một đội.

- **Thuật toán Áp dụng (Sử dụng Pandas):**

    1. **Tải dữ liệu:** Đọc file player_stats.csv vào DataFrame.
    2. **Tiền xử lý (Phát hiện & Sửa lỗi):** Dữ liệu ban đầu cho thấy tất cả các cột chỉ số (ví dụ: Performance_Gls) bị đọc nhầm thành dạng văn bản ('object').
    3. **Chuyển đổi Kiểu dữ liệu:** Sử dụng pd.to_numeric(column, errors='coerce') để ép kiểu các cột về dạng số. Những giá trị không phải số sẽ được chuyển thành NaN (Not a Number).
    4. **Áp dụng Thuật toán "Split-Apply-Combine":**
        - **Split (Tách):** Sử dụng df.groupby('Squad') để chia DataFrame lớn thành các nhóm nhỏ hơn, mỗi nhóm là một đội.

- **Apply (Áp dụng):** Trên mỗi nhóm, áp dụng hàm .agg(['mean', 'median', 'std']) để tính toán đồng thời cả ba chỉ số.
- **Combine (Kết hợp):** Pandas tự động kết hợp kết quả của tất cả các đội thành một DataFrame duy nhất.

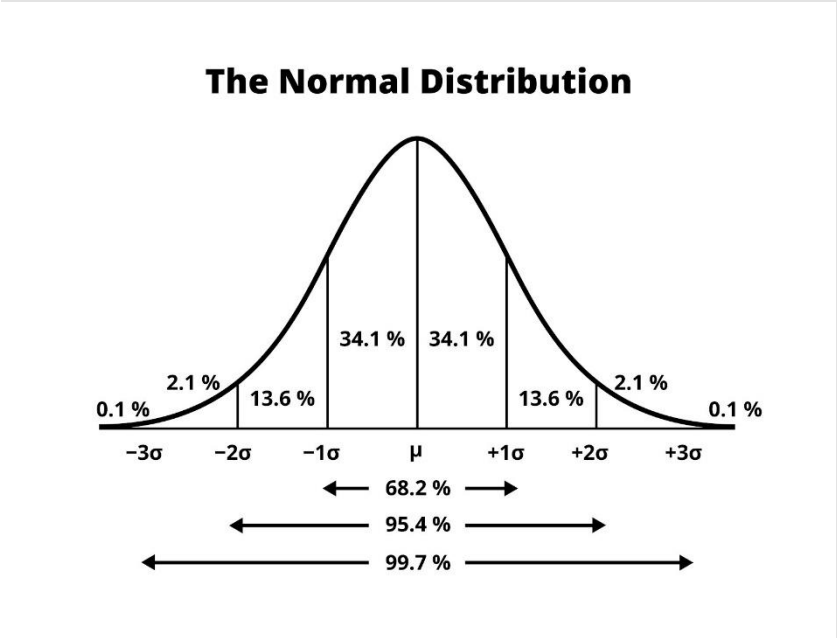
- Kết quả Đạt được:
  Kết quả được lưu vào file team_stats_summary.csv. File này cung cấp một cái nhìn tổng quan về "profile" của từng đội (ví dụ: đội nào có độ lệch chuẩn bàn thắng cao, cho thấy sự phụ thuộc vào một vài siêu sao).

2. Đội bóng có chỉ số cao nhất & Nhận định Phong độ

- Lý giải cách làm:
  Sử dụng file squad_stats.csv (đã tổng hợp sẵn chỉ số của đội) để tìm ra đội xuất sắc nhất (giá trị cao nhất) cho từng chỉ số.
- **Thuật toán Áp dụng:**
  - Tải file squad_stats.csv.
  - Lặp qua từng cột chỉ số.
  - Sử dụng hàm .idxmax() để tìm tên đội ('Squad') ứng với giá trị lớn nhất trong cột đó.
  - (Tùy chọn) Sử dụng .idxmin() cho các chỉ số tiêu cực (ví dụ: tìm đội ít thẻ vàng nhất).
- **Kết quả & Nhận định:**
  - **Liverpool:** Dẫn đầu tuyệt đối về mặt tấn công, đứng đầu ở các chỉ số: **Bàn thắng (Gls)**, **Kiến tạo (Ast)**, **Bàn thắng kỳ vọng (xG)**, và **Kiến tạo kỳ vọng (xAG)**.
  - **Manchester City:** Dẫn đầu về khả năng kiểm soát và áp đặt lối chơi, đứng đầu ở: **Tỷ lệ kiểm soát bóng (Possession)**, **Chuyền tịnh tiến (PrgP)**, **Rê bóng tịnh tiến (PrgC)** và **Kỷ luật (ít thẻ vàng nhất)**.
- **Nhận định đội có phong độ tốt nhất:** Dựa trên dữ liệu, không có đội nào "tốt nhất" một cách toàn diện. **Liverpool** có phong độ *tấn công* hủy diệt nhất, trong khi **Manchester City** có phong độ *kiểm soát trận đấu* và *sự ổn định* tốt nhất.

III. Mô hình Định giá Cầu thủ (Mục III.2)

- Phương pháp Đề xuất: Định giá bằng Z-Score (Điểm Z)
  Dựa trên kết quả từ Mục II.1 (đã có mean và std của đội), chúng ta có thể xây dựng một mô hình định giá tương đối.
- Lý giải Thuật toán:
  Z-Score đo lường mức độ "nổi bật" (vượt trội hoặc kém hơn) của một cầu thủ so với mức trung bình của chính đội anh ta, tính bằng số lần độ lệch chuẩn.



Shutterstock

$$Z = \frac{X - \mu}{\sigma}$$

* X: Chỉ số của cầu thủ A.

* μ: Chỉ số `mean` (trung bình) của đội A.

* σ: Chỉ số `std` (độ lệch chuẩn) của đội A.

Quy trình thực hiện:

1. Hợp nhất dữ liệu: Merge (kết hợp) file `player_stats.csv` (đã làm sạch) với file `team_stats_summary.csv` (đã làm sạch) dựa trên cột 'Squad'.

2. Định nghĩa Trọng số (Weights): Chọn các chỉ số quan trọng (KPIs) và gán trọng số cho chúng (ví dụ: `Performance_Gls: 30%`, `Performance_Ast: 25%`, `Expected_xG: 20%`, v.v.).

3. Tính Z-Score:** Tính Z-Score cho cầu thủ ở từng KPI.

4. Tính Điểm Giá trị (Valuation Score): Lấy tổng có trọng số của các Z-Score riêng lẻ.

   `ValuationScore = (ZScore_Gls * 0.30) + (ZScore_Ast * 0.25) + ...`

- Kết quả Đạt được:
  Một file player_valuations.csv được tạo ra, xếp hạng tất cả các cầu thủ dựa trên ValuationScore. Những cầu thủ có điểm cao nhất là những người "gánh team" hiệu quả nhất, đóng góp vượt xa mức trung bình của đồng đội.

IV. Phân cụm và Trực quan hóa Cầu thủ (Mục III)

1. Phân loại Cầu thủ bằng K-Means

- Lý giải cách làm:
  Sử dụng thuật toán học không giám sát K-Means để tự động nhóm các cầu thủ có các chỉ số (phong cách chơi) tương tự nhau vào cùng một cụm.

- **Thuật toán Áp dụng:**

  1. **Chọn Chỉ số (Features):** Lọc ra một bộ các chỉ số quan trọng để phân loại (ví dụ: Age, Per 90 Minutes_Gls, Per 90 Minutes_Ast, Progression_PrgC, Progression_PrgP, Performance_CrdY...). Việc sử dụng chỉ số "Per 90 Minutes" là rất quan trọng để chuẩn hóa thời gian thi đấu.

  2. **Tiền xử lý:** Loại bỏ các cầu thủ có dữ liệu NaN hoặc có thời gian thi đấu quá ít (ví dụ: dưới 5 trận * 90 phút).

  3. **Co giãn Dữ liệu (Scaling):** *Bước bắt buộc* với K-Means. Sử dụng StandardScaler để đưa tất cả các chỉ số về cùng một thang đo (Z-score). Điều này ngăn các chỉ số có giá trị lớn (như Progression_PrgP) lấn át các chỉ số có giá trị nhỏ (như Per 90 Minutes_Gls).

  4. **Huấn luyện Mô hình:** Chạy KMeans(n_clusters=K) trên dữ liệu đã co giãn.

- Kết quả Đạt được:
  Một file player_clusters.csv được tạo ra, trong đó mỗi cầu thủ được gán một nhãn Cụm (ví dụ: 0, 1, 2, 3, 4).

2. Xác định Số cụm (K) Tối ưu

- Lý giải cách làm:
  Để trả lời câu hỏi "Nên phân loại cầu thủ thành bao nhiêu nhóm?", chúng ta cần sử dụng các phương pháp toán học để tìm ra số K tối ưu, thay vì đoán mò.

- **Thuật toán Áp dụng:**

  1. **Biểu đồ Elbow (Khuỷu tay):**

     - **Cách làm:** Chạy K-Means với nhiều giá trị K (ví dụ: từ 1 đến 10). Vẽ biểu đồ K so với Inertia (tổng bình phương khoảng cách đến tâm cụm).

     - **Cách đọc:** Tìm điểm "khuỷu tay", nơi đường cong bắt đầu phẳng ra.

     - **Kết quả:** Biểu đồ cho thấy một điểm khuỷu tay tiềm năng ở K=4 hoặc K=5.

  2. **Biểu đồ Silhouette:**

     - **Cách làm:** Vẽ biểu đồ K (bắt đầu từ 2) so với Silhouette Score (điểm đo độ tách biệt và chặt chẽ của cụm).

     - **Cách đọc:** Tìm K có điểm Silhouette cao nhất (gần 1 nhất).

     - **Kết quả:** Biểu đồ cho thấy điểm cao nhất (peak) rõ ràng tại **K=5**.

- Nhận xét Kết quả (Nên chọn K=5):
  Dựa trên việc Biểu đồ Silhouette đạt đỉnh tại K=5 và K=5 cũng là một điểm "khuỷu tay" hợp lý trên Biểu đồ Elbow, chúng ta kết luận K=5 là số nhóm tối ưu. Con số 5 rất có ý nghĩa trong bóng đá, có thể đại diện cho các vai trò chính như: Tiền đạo cắm, Tiền vệ sáng tạo, Cầu thủ chạy cánh, Tiền vệ phòng ngự/Hậu vệ, v.v.

3. Giảm chiều và Trực quan hóa bằng PCA

- Lý giải cách làm:
  Dữ liệu của chúng ta có nhiều chiều (9 chỉ số), không thể vẽ lên biểu đồ 2D hay 3D. Chúng ta sử dụng PCA (Phân tích Thành phần Chính) để "nén" 9 chiều này xuống 2 hoặc 3 chiều, trong khi vẫn cố gắng giữ lại nhiều thông tin (phương sai) nhất có thể. Mục đích là để trực quan hóa 5 cụm K-Means đã tìm thấy.

- **Thuật toán Áp dụng:**
  - Sử dụng dữ liệu đã được co giãn (StandardScaler) từ bước K-Means.
  - Chạy PCA(n_components=2) để lấy 2 Thành phần Chính (PC1, PC2).
  - Chạy PCA(n_components=3) để lấy 3 Thành phần Chính (PC1, PC2, PC3).
  - Tạo biểu đồ scatter plot (biểu đồ phân tán) 2D (X=PC1, Y=PC2) và 3D (X=PC1, Y=PC2, Z=PC3).
  - Tô màu (c=) cho từng điểm (cầu thủ) dựa trên nhãn Cụm (Cluster) đã tìm thấy ở bước K-Means.

- Kết quả Đạt được:
  Hai biểu đồ (pca_2d_scatter.png và pca_3d_scatter.png) đã được tạo:
  - **2D:** Nắm bắt được khoảng **59.12%** tổng thông tin (phương sai) của dữ liệu.
  - **3D:** Nắm bắt được khoảng **71.53%** tổng thông tin.
  - **Nhận xét:** Cả hai biểu đồ đều cho thấy 5 cụm (5 màu) được phân tách tương đối rõ ràng. Điều này khẳng định rằng thuật toán K-Means đã tìm ra các nhóm cầu thủ thực sự khác biệt nhau về mặt thống kê. Sự chồng chéo nhẹ ở khu vực trung tâm cho thấy sự tồn tại của các cầu thủ đa năng (hybrid players) nằm giữa các vai trò.

V. Tổng kết

Từ hai file dữ liệu thô, chúng ta đã thực hiện một quy trình phân tích đầy đủ: từ thống kê mô tả tổng quan (Mục II.1), nhận định hiệu suất đội (II.2), đề xuất mô hình định giá cầu thủ (III), đến phân loại vai trò cầu thủ bằng K-Means (IV.1), chứng minh lựa chọn phân loại (IV.2), và cuối cùng là trực quan hóa các nhóm này bằng PCA (IV.3). Các kết quả đã được lưu vào nhiều file CSV và hình ảnh, cung cấp một bộ công cụ phân tích đa chiều về các cầu thủ và đội bóng.